

Fabric-as-Code

Deploy Microsoft Fabric end-to-end — from nothing to a running platform

Azure CLI · Bicep · Fabric REST APIs

The problem: two control planes

Layer	What it manages	Tooling here
Azure	Fabric capacity (Microsoft.Fabric/capacities) — billable compute	Bicep + az
Fabric	Workspaces + items (Lakehouse, Warehouse, Notebook, Pipeline)	Fabric REST
Data	Objects inside an item — e.g. stored procedures	T-SQL

The building blocks — what each part does

Component	What it is / does
Capacity (F-SKU)	The compute every item runs on. Billed hourly; pause to stop.
Workspace	Logical container for items; must sit on a capacity.
Lakehouse	OneLake files + Delta tables; auto SQL endpoint (read-only T-SQL).
Warehouse	Full read/write T-SQL engine: tables, views, stored procs, transactions.
Notebook	Spark/PySpark compute; bound to a Lakehouse to read/write tables.
Data Pipeline	Orchestrator chaining notebooks, copies and stored procedures.

What the automation does (6 steps)

- Provision resource group + Fabric capacity — 02 → `infra/capacity.bicep`
- Create the workspace — 03 → `POST /v1/workspaces`
- Assign workspace to capacity — 04 → `assignToCapacity` (unlocks items)
- Deploy items — 05 → Lakehouse, Warehouse, Notebook, Pipeline
- Deploy stored procedures — 06 → T-SQL to the Warehouse SQL endpoint
- Optional: connect to Git — 07 → workspace under source control
- Run individually to learn, or all at once with `deploy-all`.

The deployment scripts (PowerShell + bash)

Script	What it does
00-prerequisites	Checks az, pwsh/bash, sqlcmd, and login state
01-login	az login — interactive or service principal
02-provision-capacity	Bicep deploy of the Fabric capacity
03 / 04	Create workspace · assign it to the capacity
05-deploy-items	Create the four items from definition files
06-deploy-stored-procedures	Run T-SQL against the Warehouse (Entra token)
99-teardown	Delete the workspace and the resource group

How it stays reliable

- Idempotent — existing workspaces/items are detected and reused, not duplicated
- State — resolved GUIDs saved to .state.json between steps
- Auth — one az session mints tokens for Fabric and SQL; no extra tooling for procs
- Long-running ops — REST helper polls Operation-Location until Succeeded
- Templated definitions — __TOKENS__ in notebook/pipeline JSON become real GUIDs
- Portable — same definitions deploy into any workspace/tenant

Repeatable across tenants & RGs

- All tenant / subscription / secret values live in a git-ignored .env
- Duplicate per environment: .env.dev, .env.customerA, ...
- Same scripts, different config → identical environment anywhere
- Service-principal auth for CI/CD and multi-tenant automation
- Public repo: contains no secrets

Fabric — workspace contents

Every item created via the Fabric REST API (Lakehouse + SQL endpoint, Warehouse, Notebook, Pipeline). Created by scripts 03–05.

 **Fabric-as-Code Demo** 

Workspace provisioned end-to-end by the fabric-as-code repo.

+ New item

 New folder

 Import ▾

 Migrate

	Name	Status	Type	Task	Owner	Refresh
	lh_demo		Lakehouse	—	MOD Administrator	—
	lh_demo		SQL analytics endpoint	—	MOD Administrator	—
	nb_demo_load		Notebook	—	MOD Administrator	—
	pl_demo_ingest		Pipeline	—	MOD Administrator	—
	wh_demo		Warehouse	—	MOD Administrator	—

Fabric — Lakehouse with data

orders_bronze Delta table (5 rows), written by the notebook run — the bronze landing layer. The auto SQL endpoint enables read-only T-SQL / BI.

lh_demo

Search

Home

Materialized lake views

Get data

New semantic model

Add to data agent

Manage OneLake security

Update all variables

For optimal refresh to work efficiently, enable the delta CDF property on both source tables and materialized lake views used as source.

A SQL analytics endpoint for SQL querying was created with this item.

Explorer

Add lakehouses

lh_demo

Tables

orders_bronze

Files

orders_bronze

Table view

	12L order_id	12L customer_id	12L order_date	12 amount	ABC status
1	1	101	2026-01-05	120.5	shipped
2	2	102	2026-01-06	75	shipped
3	3	101	2026-01-07	240	pending
4	4	103	2026-01-08	18.99	cancelled
5	5	104	2026-01-09	999.95	shipped

Analyze your data

Explore your lakehouse data in a notebook, SQL analytics endpoint, or eventhouse endpoint.

Analyze data with

Fabric — Notebook

nb_demo_load: PySpark builds the dataset and saveAsTable writes it to the attached Lakehouse. The Lakehouse binding is injected at deploy time.

The screenshot displays the Microsoft Fabric Notebook interface. At the top, the notebook name 'nb_demo_load' is shown with a dropdown arrow. A search bar is located to the right. Below this is a navigation bar with tabs: 'Home', 'Edit', 'AI tools', 'Run', and 'View'. A secondary bar contains icons for file operations, a 'Run all' button, a 'Connect' button, a language selector set to 'PySpark (Python)', an 'Environment' dropdown set to 'Workspace default', a 'Data Wrangler' button, and icons for Copilot and other tools.

The left sidebar, titled 'Explorer', shows a tree view with 'Data items', 'Resources', and 'Connections'. Under 'Data items', there is a '+ Add data items' button and a section for 'OneLake'.

The main content area features a warning banner: 'Other people in your organization may have access to notebooks and Spark job definitions in this workspace. Carefully review this item before running it.' Below the banner, the notebook title 'Demo load notebook' is displayed, followed by a description: 'Provisioned by fabric-as-code. Writes a small demo dataset into the attached Lakehouse as a Delta table.'

The notebook contains two code blocks. The first block defines a PySpark DataFrame with five rows of order data. The second block writes the DataFrame to a Delta table in the Lakehouse and prints a confirmation message.

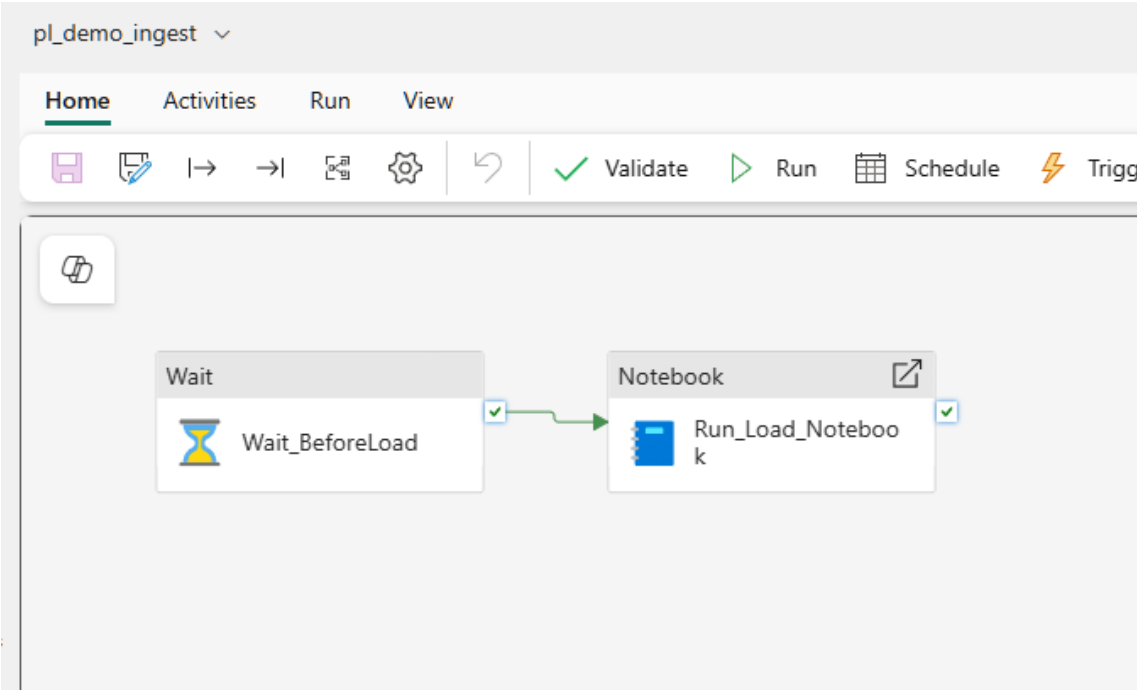
```
1 from pyspark.sql import Row
2 from datetime import date
3
4 rows = [
5     Row(order_id=1, customer_id=101, order_date=date(2026, 1, 5), amount=120.50, status='shipped'),
6     Row(order_id=2, customer_id=102, order_date=date(2026, 1, 6), amount=75.00, status='shipped'),
7     Row(order_id=3, customer_id=101, order_date=date(2026, 1, 7), amount=240.00, status='pending'),
8     Row(order_id=4, customer_id=103, order_date=date(2026, 1, 8), amount=18.99, status='cancelled'),
9     Row(order_id=5, customer_id=104, order_date=date(2026, 1, 9), amount=999.95, status='shipped'),
10 ]
11 df = spark.createDataFrame(rows)
12 df.show()
```



```
1 df.write.mode('overwrite').format('delta').saveAsTable('orders_bronze')
2 print('Wrote orders_bronze to the Lakehouse.')
```

Fabric — Data Pipeline

Design: Wait → Run Notebook (success dependency). Run: Succeeded. Orchestration that executed the notebook and populated the Delta table.



Recent runs for pl_demo_ingest

Activity name	Start time	Submitted by	Status
pl_demo_ingest	06/26/2026, 12:42 PM	MOD Administrator	✓ Succeeded

Azure — Fabric capacity

The billable Microsoft.Fabric/capacities resource (F2) — the compute all items run on. Deployed by Bicep; pause to stop hourly billing.

 **fabasc93lr8u**
Fabric Capacity



Diagnose issues with this resource

Show performance trends for this resource

What metric dimensions are supported for this resource

Search

⌵ ⌵

⏸ Pause

🔄 Refresh

➡ Move ⌵

🗑 Delete

Overview

Activity log

Access control (IAM)

Tasks

Essentials

Resource group [\(move\)](#) : [rg-fabric-as-code](#)

Status : Active

Location : West US 3

Resource name : fabasc93lr8u

SKU : F2

What you can build on this

- Medallion platform — Lakehouse bronze/silver/gold + Spark notebooks
- SQL warehousing — Warehouse with versioned schema, tables & stored procedures
- Orchestration — Pipelines chaining notebooks, copies & procedures
- Promotion — Git integration + deployment pipelines (dev → test → prod)
- Governance as code — capacity sizing, admins & RBAC in source control

Get started

- `git clone https://github.com/<owner>/fabric-as-code`
- `cp .env.example .env` (fill in tenant / subscription / capacity)
- `pwsh ./scripts/powershell/deploy-all.ps1`
- ...or `./scripts/bash/deploy-all.sh`
- Tear it all down with `99-teardown`.